NORTHSTAR TECHNOLOGIES
PVT. LTD.

API DEVELOPMENT STANDARDS

Backend engineering guidelines for secure, consistent and maintainable APIs

Document ID	ENG-GDE-010
Document Type	GUIDE
Access Scope	TEAM_ONLY
Department	Backend Engineering
Version	1.0
Effective Date	04 November 2026
Document Owner	Backend Engineering
Review Cycle	At least annually and after significant API/security changes

Purpose. This guide defines baseline engineering practices for designing, implementing, documenting, securing, testing and operating APIs developed or maintained by the Backend Engineering team at Northstar Technologies Pvt. Ltd. It is intended to support predictable integration between frontend applications, backend services, internal workers and approved external systems.

Fictional demonstration document for the OnBoardIQ sample knowledge base. The examples are illustrative and are not a formally approved corporate security policy or legal advice.

1. Scope and applicability

These standards apply to REST and HTTP APIs built, exposed or maintained by Backend Engineering, including service-to-service endpoints and APIs consumed by web, mobile, internal automation and AI services.

The document is restricted to the Backend Engineering department through the TEAM_ONLY access scope. Access to this guide does not itself grant access to API credentials, source repositories, environments or production systems.

- Apply these standards to new APIs and material changes to existing APIs.
- Document approved deviations with an owner, rationale, risk assessment and review date.
- Follow stricter platform, regulatory, contractual or security requirements where they apply.

2. API design principles

- **Consistency:** use predictable resource names, response structures, status codes and error formats.
- **Security by default:** authenticate callers, authorize every protected operation and minimize exposed data.
- **Explicit contracts:** define request and response schemas and document behavioural expectations.
- **Backward compatibility:** avoid breaking existing consumers without a planned versioning and migration process.
- **Reliability:** use timeouts, bounded retries, idempotency where needed and controlled resource consumption.
- **Observability:** provide structured logs, metrics and trace context without leaking sensitive information.

3. Resource naming and HTTP methods

For REST-style APIs, model endpoints around resources and use HTTP methods according to their intended semantics. Use a consistent base path, such as /api/v1, unless a platform-specific convention has been approved.

- Use plural, lowercase, kebab-case resource paths where appropriate, for example /api/v1/employee-records.
- Use GET to retrieve resources, POST to create or initiate an operation, PUT to replace, PATCH to partially update and DELETE to remove where permitted.
- Use nouns for resource paths; avoid embedding action verbs unless the endpoint represents a genuine command or operation.
- Use query parameters for filtering, sorting, pagination and optional field selection.
- Do not expose database table names or internal implementation details as public API contracts.
- Keep paths and parameters consistent across services.

4. Request and response conventions

- Use JSON for standard request and response bodies unless another content type is justified and documented.
- Set and validate Content-Type and Accept headers where relevant.
- Use consistent field naming within a service contract, preferably camelCase for JSON properties unless an established contract dictates otherwise.
- Validate required fields, data types, formats, lengths, ranges and allowed enum values at the API boundary.
- Return only fields the caller is authorized to see and needs for the task.
- Represent dates and timestamps in a documented ISO 8601 format; use UTC for instants where possible.
- Do not return stack traces, SQL errors, internal hostnames, secrets or raw exception details to clients.

5. HTTP status codes

Use status codes consistently so consumers can distinguish successful operations, client errors, authentication failures and server-side failures.

- **200 OK:** successful request with a response body.
- **201 Created:** resource successfully created; include a Location header when useful.
- **202 Accepted:** request accepted for asynchronous processing.
- **204 No Content:** successful operation with no response body.
- **400 Bad Request:** malformed or invalid request.
- **401 Unauthorized:** authentication is missing or invalid.
- **403 Forbidden:** caller is authenticated but not permitted to perform the action.
- **404 Not Found:** resource does not exist or should not be disclosed to the caller.
- **409 Conflict:** request conflicts with current resource state or a uniqueness constraint.
- **422 Unprocessable Content:** syntactically valid request that fails semantic validation, where adopted by the service.
- **429 Too Many Requests:** caller exceeded a rate or quota limit.
- **500/502/503/504:** server or upstream service failures; avoid exposing internal details.

6. Error response format

Use a consistent, machine-readable error structure. A recommended illustrative shape is:

```
{"error":{"code":"VALIDATION_ERROR","message":"Request validation
failed","details":[{"field":"email","issue":"Invalid format"}],"requestId":"..."}}
```

Error codes should be stable and suitable for programmatic handling. Human-readable messages may evolve, but clients should not need to parse message text to determine behaviour.

7. Authentication and authorization

- Use the organization's approved authentication mechanism, such as secure session-based authentication or appropriately managed token-based authentication.
- Authenticate every non-public endpoint; do not treat possession of an obscure URL as authorization.
- Perform authorization checks on the server for every resource and action, including object-level access checks.
- Enforce role, department, ownership and document/data scope rules in backend services rather than relying on frontend visibility.
- Apply least privilege to service identities and separate user permissions from service-to-service permissions.
- Use secure cookie settings when cookie-based sessions are used, including HttpOnly, Secure and an appropriate SameSite policy.
- Protect state-changing cookie-authenticated requests against CSRF where applicable.
- Never place passwords, API keys, access tokens or session identifiers in URLs.

8. Input validation and API security

- Treat all client input, headers, uploaded content and upstream responses as untrusted.
- Use parameterized database queries or safe ORM/query-builder mechanisms; do not concatenate untrusted input into SQL.
- Prevent mass assignment by explicitly allowing writable fields.
- Apply output encoding and context-appropriate sanitization where data is rendered by clients.
- Restrict upload size, file types and processing resources; validate file content rather than trusting extensions alone.
- Configure CORS with explicit trusted origins and required methods/headers; avoid wildcard origins with credentialed requests.

- Use TLS for API traffic and validate certificates for outbound service calls.
- Apply rate limits, request-size limits and abuse protections to sensitive or resource-intensive endpoints.
- Keep dependencies and API frameworks patched through the approved vulnerability-management process.

9. Pagination, filtering and sorting

- Paginate collection endpoints that may return large result sets.
- Set a reasonable default page size and enforce a maximum page size on the server.
- Use stable ordering to prevent records from being skipped or repeated between pages.
- Prefer cursor-based pagination for large or frequently changing datasets when appropriate.
- Allowlist sortable fields and filter operations; never interpolate arbitrary client-provided field names into database queries.
- Document pagination parameters, response metadata and continuation behaviour.

10. Versioning and compatibility

- Use a documented versioning strategy. A URI prefix such as /api/v1 is an acceptable convention for many REST services.
- Treat removal or renaming of fields, changed validation rules, altered authorization behaviour and changed response semantics as potentially breaking changes.
- Prefer additive, backward-compatible changes where possible.
- Communicate deprecation timelines and migration guidance to affected consumers.
- Maintain old versions only for the approved support period and remove them through a communicated process.
- Keep OpenAPI specifications aligned with the deployed implementation.

11. Idempotency, retries and asynchronous work

- Design safe retry behaviour for network failures and transient upstream errors.
- Use idempotency keys for operations where duplicate submissions could create duplicate payments, records, jobs or other side effects.
- Do not retry non-idempotent operations blindly.
- Use bounded retries with backoff and jitter; avoid retry storms.
- For long-running work, consider returning 202 Accepted with a job or operation reference and a documented status endpoint.
- Ensure queue consumers and background handlers can safely handle duplicate delivery where the messaging platform permits it.

12. Timeouts, rate limits and resource controls

- Set explicit connection and request timeouts for outbound dependencies.
- Avoid unbounded waits, unbounded result sets and unbounded request payloads.
- Apply rate limits appropriate to endpoint sensitivity and workload cost.
- Return 429 responses with useful retry guidance where appropriate.
- Use circuit breakers or equivalent protection for unstable dependencies where justified.
- Define concurrency and resource limits for expensive exports, searches, AI calls and file-processing endpoints.

13. Documentation and API contracts

- Maintain an OpenAPI specification for HTTP APIs where supported.

- Document endpoint purpose, authentication, authorization requirements, parameters, request schemas, response schemas, status codes and error codes.
- Include representative examples that do not contain real personal data, secrets or production identifiers.
- Document pagination, rate limits, idempotency, versioning and deprecation behaviour.
- Update API documentation in the same change that updates implementation behaviour.
- Use contract review to identify breaking changes before release.

14. Logging, monitoring and tracing

- Use structured logs with timestamp, service name, environment, severity and correlation/request ID.
- Record relevant request outcomes and latency while avoiding unnecessary payload capture.
- Never log passwords, bearer tokens, session cookies, private keys or other secrets.
- Mask or omit sensitive personal and confidential fields from logs.
- Track request volume, error rates, latency percentiles, saturation and dependency health.
- Propagate trace or correlation context across service boundaries where supported.
- Restrict access to logs and apply approved retention controls.

15. Testing requirements

- Write unit tests for business logic, validation and error handling.
- Add integration tests for database and dependent-service interactions.
- Test authentication, authorization and object-level access boundaries, including negative cases.
- Test malformed inputs, oversized payloads, rate limits, timeouts and upstream failures.
- Use contract tests or schema validation to detect unintended API incompatibilities.
- Use synthetic or appropriately masked data in non-production environments.
- Include regression tests for fixed security and reliability defects.

16. Deployment and release checklist

- API contract and OpenAPI documentation are updated.
- Authentication and authorization rules have been reviewed and tested.
- Input validation, output filtering and rate limits are in place.
- Automated tests and relevant security checks pass.
- Database migrations and dependent-service changes are coordinated.
- Configuration and secrets are supplied through approved secure mechanisms.
- Timeouts, health checks, logs, metrics and alerts are configured.
- Rollback or recovery steps and consumer communication are prepared.
- The change has required code review and release approval.

17. Ownership and responsibilities

Backend engineers: implement and maintain API contracts, business logic, validation, authorization, tests and technical documentation.

Backend technical leads / service owners: review architecture, compatibility, security-sensitive changes and production readiness.

Consuming teams: use documented contracts, handle errors and deprecations correctly, and avoid depending on undocumented behaviour.

DevOps / IT Operations: support approved deployment, infrastructure, monitoring and incident processes.

Specific approval authority must follow Northstar's actual service ownership and release governance.

18. Exceptions and incident handling

Exceptions must be documented with the affected API, reason, owner, risk, compensating controls and expiry or review date. Obtain approval through the applicable engineering and security governance process.

Report suspected credential exposure, unauthorized API access, sensitive-data leakage, abnormal traffic or service compromise promptly through the approved incident channel. Preserve relevant logs and evidence and avoid unapproved destructive remediation.

19. Glossary

- **API:** an interface through which software systems exchange data or invoke capabilities.
- **OpenAPI:** a specification format for describing HTTP API contracts.
- **Idempotency:** a property where repeating an operation has the same intended effect as performing it once.
- **CORS:** browser mechanism controlling cross-origin requests.
- **CSRF:** cross-site request forgery, an attack that tricks a browser into making an unintended authenticated request.
- **Rate limiting:** restricting the number or cost of requests within a defined period.
- **Correlation ID:** an identifier used to associate logs and events belonging to a request or workflow.

Document review and governance

Backend Engineering should review this guide at least annually and whenever API platform architecture, security requirements or integration patterns materially change. Updates must be versioned and communicated to affected consumers.

End of document. Fictional Northstar Technologies Pvt. Ltd. demonstration artifact for the OnBoardIQ knowledge base. All procedures and examples are illustrative.